

MENTIONS_USER_GUIDE

@ Mentions User Guide

HelixCode Context Injection System

Version: 1.0 Last Updated: November 7, 2025 Feature Status:  Production Ready

Table of Contents

- 1. [Introduction](#)
- 2. [Quick Start](#)
- 3. [Mention Types](#)
- 4. [Syntax Reference](#)
- 5. [Advanced Usage](#)
- 6. [Best Practices](#)
- 7. [Examples](#)
- 8. [Troubleshooting](#)
- 9. [FAQ](#)

Introduction

The @ Mentions system in HelixCode allows you to inject context from various sources directly into your conversation with the AI. Instead of copying and pasting file contents or describing your codebase, you can use concise @ mentions to provide the AI with exactly the context it needs.

Benefits

- **Time Saving:** No more copy-pasting file contents
- **Accuracy:** AI sees actual file contents, not your description
- **Efficiency:** Smart token management prevents context overflow
- **Convenience:** Fuzzy search finds files even with partial names
- **Flexibility:** 7 different mention types for different contexts

Supported Mention Types

Mention	Purpose	Example
@file	Include file contents	@file[src/main.go]
@folder	List folder contents	@folder[internal] (recursive=true)
@url	Fetch web content	@url[https://docs.example.com]

Mention	Purpose	Example
@git-changes	Show uncommitted changes	@git-changes
@[commit]	Show specific commit	@[abc123]
@terminal	Include terminal output	@terminal(lines=50)
@problems	Show workspace errors	@problems(type=errors)

Quick Start

Basic File Mention

Can you review the authentication logic in @file[internal/auth/handler.go]

The AI will receive the full contents of handler.go along with your question.

Fuzzy File Search

Fix the bug in @file[auth handler]

HelixCode will fuzzy search for files matching “auth handler” and use the best match (e.g., internal/auth/handler.go).

Multiple Mentions

Compare @file[old/implementation.go] with @file[new/implementation.go] and

You can use multiple @ mentions in a single message.

Mention Types

1. @file - File Contents

Purpose: Include the complete contents of a source file.

Syntax:

```
@file[path/to/file.go]
@file[partial filename]
@file[*.*go] (max_lines=100)
```

Examples:

Basic usage:

```
@file[main.go]
```

Fuzzy search:

```
@file[auth handler]
# Matches: internal/auth/handler.go
```

With line limit:

```
@file[large_file.go](max_lines=200)
```

Features: -  Absolute and relative path support -  Fuzzy file name matching -  Automatic syntax detection -  Token counting and truncation -  Binary file detection

Supported Options: - `max_lines`: Maximum lines to include (default: entire file)

2. @folder - Folder Contents

Purpose: List files in a directory, optionally with file contents.

Syntax:

```
@folder[path/to/dir]
@folder[path](recursive=true)
@folder[path](content=true)
@folder[path](recursive=true,content=true)
```

Examples:

List files in folder:

```
@folder[internal/auth]
```

Recursive listing:

```
@folder[src](recursive=true)
```

Include file contents:

```
@folder[config](content=true)
```

Both recursive and content:

```
@folder[internal/llm](recursive=true,content=true)
```

Features: -  Recursive and non-recursive modes -  Optional file content inclusion - 
Respects .gitignore patterns -  Smart token budget management -  Filters out binary/
generated files

Supported Options: - `recursive`: Walk subdirectories (default: false) - `content`: Include file contents (default: false)

Output Format:

```
 subdir/
 file1.txt (245 bytes)
 file2.go (1024 bytes)
Content:
```

package main ...

3. @url - Web Content

Purpose: Fetch and include content from a web URL.

Syntax:

```
@url[https://example.com/page]
@url[https://docs.site.com](format=text)
```

Examples:

Documentation page:

```
@url[https://pkg.go.dev/github.com/gin-gonic/gin]
```

API response:

```
@url[https://api.github.com/repos/user/repo]
```

Features: -  HTML to text conversion -  Markdown preservation -  Open Graph metadata extraction -  15-minute caching (prevents duplicate fetches) -  Automatic content-type detection

Supported Options: - `format`: Output format (text, html, markdown) - default: text

Output Includes: - Page title - URL - Content (converted to text) - Metadata (description, author if available)

Limitations: - Maximum fetch size: 1MB - Timeout: 30 seconds - JavaScript-rendered content may not be captured

4. @git-changes - Uncommitted Changes

Purpose: Show all uncommitted changes in the Git repository.

Syntax:

```
@git-changes
@git-changes(staged=true)
@git-changes(unstaged=true)
```

Examples:

All changes:

```
Review my changes: @git-changes
```

Only staged changes:

```
@git-changes(staged=true)
```

Only unstaged changes:

```
@git-changes(unstaged=true)
```

Features: -  Shows diff output (git diff) -  Includes file additions/deletions -  Filters by staged/unstaged -  Supports both modified files and new files

Output Format:

```
diff --git a/internal/auth/handler.go b/internal/auth/handler.go
index 1234567..abcdefg 100644
--- a/internal/auth/handler.go
+++ b/internal/auth/handler.go
@@ -45,7 +45,8 @@
-     if err != nil {
+     if err != nil && err != ErrNotFound {
```

Requirements: - Must be in a Git repository - Git must be installed and in PATH

5. @[commit] - Specific Commit

Purpose: Show the diff from a specific Git commit.

Syntax:

```
@[commit-hash]
@[HEAD~3]
@[branch-name]
```

Examples:

Specific commit:

```
@[abc123def]
```

Recent commit:

```
@[HEAD~1]
```

Branch:

```
@[feature/new-auth]
```

Features: -  Full commit diff -  Commit metadata (author, date, message) -  Supports commit references (HEAD~N, branch names) -  Shows all changed files

Output Includes: - Commit hash - Author and date - Commit message - Full diff

Requirements: - Must be in a Git repository - Commit must exist in history

6. @terminal - Terminal Output

Purpose: Include recent terminal output in the conversation.

Syntax:

```
@terminal
@terminal(lines=50)
@terminal(lines=100)
```

Examples:

Last 100 lines (default):

The build failed, see @terminal

Last 50 lines:

```
@terminal(lines=50)
```

Features: -  Captures stdout and stderr -  Preserves ANSI colors (converted to text descriptions) -  Configurable line count -  Automatic terminal history management

Supported Options: - `lines`: Number of lines to include (default: 100, max: 1000)

Use Cases: - Debugging build errors - Sharing test output - Analyzing log output - Troubleshooting command failures

Note: Terminal history is session-specific and cleared on restart.

7. @problems - Workspace Problems

Purpose: Show errors and warnings from the workspace.

Syntax:

```
@problems
@problems(type=errors)
@problems(type=warnings)
@problems(type=all)
```

Examples:

All problems:

Fix these issues: @problems

Only errors:

```
@problems(type=errors)
```

Only warnings:

```
@problems(type=warnings)
```

Features: -  Integrates with language servers (LSP) -  Shows file, line, and column - 
Categorized by severity -  Includes problem descriptions

Supported Options: - type: Problem filter (errors, warnings, all) - default: all

Output Format:

 ERROR internal/auth/handler.go:45:12
undefined: validateToken

 WARNING internal/llm/provider.go:123:5
variable 'timeout' is unused

 INFO config/config.go:67:1
consider using a constant for this value

Syntax Reference

Basic Syntax

```
@mention-type[target]  
@mention-type[target] (option1=value1)  
@mention-type[target] (option1=value1,option2=value2)
```

Components

Mention Type: The type of content to include (file, folder, url, etc.)

Target (optional): The path, URL, or identifier - Enclosed in square brackets [. . .] - Can be omitted for some types (e.g., @git-changes, @terminal, @problems)

Options (optional): Key-value pairs for customization - Enclosed in parentheses (. . .) - Comma-separated - Format: key=value

Examples

@file[main.go]	# Simple file
@folder[src]	# Simple folder
@folder[src] (recursive=true)	# Folder with option
@folder[src] (recursive=true,content=true)	# Multiple options
@url[https://docs.example.com]	# URL
@terminal(lines=50)	# No target, just option
@problems(type=errors)	# No target, just option

Advanced Usage

Combining Multiple Mentions

You can use multiple @ mentions in a single message to provide comprehensive context:

I'm refactoring the authentication system.
Current implementation: @file[internal/auth/handler.go]
Current tests: @file[internal/auth/handler_test.go]
Recent changes: @git-changes
Current issues: @problems(type=errors)

Please suggest improvements and help me fix the failing tests.

Fuzzy Search Examples

The fuzzy file search is intelligent and scores matches based on: 1. Exact filename match (highest score) 2. Path contains query 3. Filename contains query 4. Sequential character match 5. Word boundary match

@file[auth]	→ internal/auth/handler.go
@file[auth test]	→ internal/auth/handler_test.go
@file[llm prov]	→ internal/llm/provider.go
@file[main]	→ cmd/server/main.go
@file[cfg]	→ config/config.go

Token Management

HelixCode automatically manages token counts to prevent context overflow:

Default Limits: - @file: Entire file (truncated if > 10k tokens) - @folder: 8000 tokens max - @folder (content=true): Files added until limit reached - @url: 5000 tokens max - @git-changes: No limit (usually small) - @terminal: Based on line count - @problems: No limit (usually small)

Override Token Limits:

@folder[large-dir](content=true,max_tokens=15000)

Best Practices

1. Be Specific with File Paths

Good:

@file[internal/auth/jwt.go]

Acceptable (if unique):

@file[jwt]

Avoid (ambiguous):

@file[auth] # Could match multiple files

2. Use Recursive Folder Scanning Wisely

Good (specific, small directory):

```
@folder[internal/auth](recursive=true)
```

Avoid (too broad):

```
@folder[.](recursive=true,content=true)  # Entire workspace!
```

3. Combine Mentions for Context

Instead of:

Can you review `internal/auth/handler.go` and `internal/auth/handler_test.go`?

Do this:

Review `@file[internal/auth/handler.go]` and `@file[internal/auth/handler_test.go]`

4. Use Terminal History for Debugging

When asking about errors:

The build fails with strange errors. Here's the output: `@terminal(lines=50)`

What's causing this?

5. Filter Problems by Type

For focused debugging:

Let's fix the errors first: `@problems(type=errors)`

6. Cache Web Content

If repeatedly referencing the same URL, the 15-minute cache prevents duplicate fetches:

Per the docs at `@url[https://docs.example.com]`, should I...

(later in same session)

Also from `@url[https://docs.example.com]`, how do I... # Uses cache

Examples

Example 1: Code Review Request

Please review this authentication implementation:

Implementation: `@file[internal/auth/handler.go]`

Tests: `@file[internal/auth/handler_test.go]`

Recent changes: `@git-changes`

Are there any security issues or improvements you'd suggest?

Example 2: Bug Investigation

I'm getting test failures. Here's the context:

Failed test file: @file[internal/llm/provider_test.go]
Implementation: @file[internal/llm/provider.go]
Test output: @terminal(lines=100)
Workspace problems: @problems(type=errors)

What's wrong and how can I fix it?

Example 3: Refactoring Assistance

I want to refactor the configuration system:

Current config: @folder[config](content=true)
Usage example: @file[internal/config/loader.go]
Documentation: @url[https://github.com/spf13/viper]

How should I restructure this to use Viper?

Example 4: API Integration

Help me integrate this API:

API Docs: @url[https://api.example.com/docs]
Current implementation: @file[internal/client/api.go]
Failing tests: @file[internal/client/api_test.go]
Error output: @terminal(lines=50)

What am I doing wrong?

Example 5: Documentation Update

Update the README to reflect these changes:

Current README: @file[README.md]
New features: @git-changes
Project structure: @folder[.](recursive=false)

Please rewrite the README to include the new features.

Troubleshooting

File Not Found

Problem:

Error: file not found: internal/auth/handler.go

Solutions: 1. Check the file path is correct (relative to workspace root) 2. Use fuzzy search: `@file[auth handler]` 3. List the folder first: `@folder[internal/auth]` 4. Verify file exists: `ls internal/auth/`

Fuzzy Search Returns Wrong File

Problem: `@file[handler]` returns `internal/llm/handler.go` instead of `internal/auth/handler.go`

Solutions: 1. Be more specific: `@file[auth handler]` 2. Use full path: `@file[internal/auth/handler.go]` 3. Check scoring with multiple attempts

URL Fetch Timeout

Problem:

Error: timeout fetching `https://slow-site.com`

Solutions: 1. Check internet connection 2. Try again (may be temporary) 3. Use alternative documentation source 4. Download page manually and use `@file`

Git Commands Fail

Problem:

Error: not a git repository

Solutions: 1. Initialize Git: `git init` 2. Navigate to repository root 3. Check Git is installed: `git --version`

Token Limit Exceeded

Problem:

Warning: content truncated (exceeded token limit)

Solutions: 1. Use smaller folders: `@folder[specific-dir]` instead of `@folder[.]` 2. Disable content: `@folder[dir](content=false)` 3. Increase limit: `@folder[dir](content=true,max_tokens=15000)` 4. Split into multiple messages

Terminal History Empty

Problem: `@terminal` returns no output

Causes: - New session (no terminal history yet) - Terminal history cleared - No commands run in this session

Solutions: 1. Run some commands first 2. Use `@file` for log files instead 3. Copy output manually if needed

FAQ

Q: Can I use @ mentions in any message? A: Yes, @ mentions work in all chat messages to HelixCode.

Q: How many @ mentions can I use in one message? A: Unlimited, but watch the total token count. Typically 5-10 mentions per message is reasonable.

Q: Do @ mentions work with all LLM providers? A: Yes, @ mentions are processed before sending to any LLM provider.

Q: Can I create custom mention types? A: Not yet, but this is planned for a future release. Currently, only the 7 built-in types are supported.

Q: Are @ mentions case-sensitive? A: File paths are case-sensitive on Linux/Mac, case-insensitive on Windows. Mention types themselves are case-insensitive.

Q: Can @ mentions include binary files? A: Binary files are detected and skipped. Only text files are included.

Q: Do @ mentions slow down responses? A: Slightly, but the delay is minimal (typically < 100ms per mention). File I/O and URL fetching are cached.

Q: Can I use @ mentions in slash commands? A: Yes! For example: `/newtask Fix bugs in @file[handler.go]`

Q: What happens if a mentioned file changes during the conversation? A: The content is read fresh each time you mention it. If you want a snapshot, copy the content in your message.

Q: Can @ mentions access files outside the workspace? A: No, all file/folder mentions are restricted to the workspace root for security.

Q: Is there a performance impact with many @ mentions? A: Minimal. HelixCode uses caching and efficient file I/O. Even 10+ mentions process quickly.

See Also

- [Slash Commands User Guide](#)
- [HelixCode Configuration Guide](#)
- [API Documentation](#)
- [Video Tutorials](#)

Document Version: 1.0 **Last Updated:** November 7, 2025 **Feedback:** [GitHub Issues](#)

Sources verified

Sources verified 2026-05-29: `https://pkg.go.dev/github.com/gin-gonic/gin` (the `@url[https://pkg.go.dev/github.com/gin-gonic/gin]` example resolves to a live, current pkg.go.dev page — the documented @url fetcher target is valid), `https://github.com/spf13/viper` (live repository — the documentation-mention example resolves). Confirmed: the @url, @file, and terminal-history mention syntaxes are HelixCode-internal

CLI features documented against the product itself, not against an external service contract.

Negative findings: (1) the example URLs `https://docs.example.com`, `https://example.com/page`, `https://api.example.com/docs` are intentional RFC-2606 `example.com` placeholders, not real endpoints — correctly illustrative; (2) the Feedback link `https://github.com/user/helixcode/issues` is a placeholder `user/ org` (the real HelixCode repos live under `vasic-digital/*` and `HelixDevelopment/*` per CLAUDE.md project-tightening note) — flagged as a stale placeholder, left unchanged to avoid guessing the canonical issues URL (§11.4.6 no-guessing).